

Greedy algorithms

COMPLETE LEARNING GUIDE

What You'll Learn

A greedy algorithm builds a solution incrementally by always making the choice that looks best right now, without reconsidering that choice later — and the entire skill of this topic is knowing precisely which problems this locally-optimal strategy actually solves correctly versus which ones it silently gets wrong. This guide covers the two properties (greedy choice property and optimal substructure) that must both hold for a greedy algorithm to be provably correct, worked examples of activity selection (interval scheduling) and coin change (with an explicit warning about when the greedy coin-change approach fails), a brief introduction to Huffman coding as a real-world greedy application, the exchange-argument technique for informally justifying why a greedy choice is safe, and a direct comparison against dynamic programming — since confusing when to reach for greedy versus DP is one of the most common conceptual errors at this level.

1. The Greedy Choice Property and Optimal Substructure

A problem is solvable by a greedy algorithm only if it has the greedy choice property — a globally optimal solution can always be reached by making a locally optimal (greedy) choice first, without ever needing to reconsider or undo that choice later — combined with optimal substructure, the same requirement DP relies on. The greedy choice property is the stricter, more fragile requirement: many problems have optimal substructure (DP works) but lack the greedy choice property (a locally best choice can lead to a globally worse outcome), which is exactly why DP is a strictly more general technique than greedy, and why greedy should be viewed as an optimization available only when you can specifically justify it, not a default first approach.

2. Worked Example: Activity Selection (Interval Scheduling)

Given a set of activities, each with a start and end time, and a single resource that can only do one activity at a time, activity selection asks for the maximum number of non-overlapping activities that can be scheduled. The greedy strategy — sort activities by end time, then repeatedly pick the next activity whose start time is not earlier than the previously selected activity's end time — is provably optimal: choosing the activity that finishes earliest always leaves the maximum possible remaining time for scheduling further activities, which is the exchange-argument intuition for why this greedy choice never costs you anything compared to any other valid first choice.

JAVA

```
// Activity Selection -- greedy by earliest finish time -- O(n log n) time
// dominated by the sort
public static int maxActivities(int[][] activities) { // each row: {start, end}
    Arrays.sort(activities, (a, b) -> a[1] - b[1]); // sort by END time
    int count = 1;
    int lastEnd = activities[0][1];

    for (int i = 1; i < activities.length; i++) {
        if (activities[i][0] >= lastEnd) { // this activity starts after (or
            // when) the last one ended
            lastEnd = activities[i][1];
            count++;
        }
    }
    return count;
}
// Time: O(n log n) -- sorting dominates; Space: O(1) extra (ignoring the sort's
// own space)
```

3. Worked Example: Coin Change — Where Greedy Works, and Where It Fails

The greedy coin-change strategy — always take the largest denomination coin that doesn't exceed the remaining amount — correctly produces the minimum number of coins for "canonical" coin systems like US currency (1, 5, 10, 25), but it FAILS for arbitrary denominations: with coins {1, 3, 4}, greedy makes change for 6 as 4+1+1 (three coins), while the actual optimal answer is 3+3 (two coins). This single counterexample is exactly why interviewers ask about coin change specifically — it's the clearest, most memorable demonstration that greedy is not universally correct, and that the correct general-purpose solution for arbitrary denominations is dynamic programming, not greedy.

JAVA

```
// Greedy coin change -- WORKS for canonical systems, WRONG for arbitrary
// denominations
public static int greedyCoinCount(int[] denominations, int amount) { //
    // denominations sorted descending
    int count = 0;
    for (int coin : denominations) {
        count += amount / coin;
        amount %= coin;
    }
    return amount == 0 ? count : -1;
}
// greedyCoinCount({4, 3, 1}, 6) -- gives 4+1+1 = 3 coins, but 3+3 = 2 coins is
// optimal
// correct general solution: dynamic programming (see the Dynamic Programming
// submodule).
```

4. Huffman Coding: A Real-World Greedy Application

Huffman coding builds an optimal prefix-free binary encoding for a set of characters based on their frequencies, by repeatedly taking the two least-frequent remaining nodes from a priority queue (min-heap) and merging them into a new combined node, until only one node — the encoding tree's root — remains. This greedy strategy is provably optimal (it minimizes the total encoded length) and is a real, still-used building block inside compression formats like ZIP and JPEG, making it one of the few classroom greedy algorithms with immediate,

direct industrial relevance rather than being purely a teaching example.

5. The Exchange Argument: Informally Justifying a Greedy Choice

An exchange argument is the standard technique for informally proving a greedy strategy correct: assume some optimal solution doesn't make the greedy choice, then show that "exchanging" that solution's first choice for the greedy choice produces another solution that's at least as good — since this can be repeated for every deviation from greedy, an optimal solution matching greedy's choices must exist. For activity selection, the exchange argument is: if an optimal schedule's first activity doesn't finish earliest, swap it for the one that does — this swap can only free up more time for the remaining schedule, never less, so the swapped schedule is still optimal. Being able to sketch this kind of argument, even informally, is what separates "I think greedy works here" from "I can show greedy works here" in an interview setting.

6. Greedy vs. Dynamic Programming: When to Reach for Which

Both techniques build a solution from smaller subproblems, but greedy commits to one choice per step and never revisits it, while DP considers all valid choices at each step and remembers the best outcome for every possible subproblem state, which is strictly more powerful but also strictly more expensive. The practical decision process: try to find and justify a greedy strategy first (often via an exchange argument or a clear intuitive reason a locally best choice can't hurt); if you can construct a counterexample where the greedy choice leads to a suboptimal overall result (as coin change with denominations {1,3,4} demonstrates), that's proof greedy doesn't apply and you need DP instead.

Common Pitfalls

- Assuming greedy works for coin change with arbitrary denominations — it only works for canonical coin systems; the general problem requires DP.
- Applying a greedy strategy without being able to justify why the locally optimal choice can never lead to a worse global outcome (no exchange-argument reasoning).
- Confusing "has optimal substructure" (necessary for both greedy and DP) with "has the greedy choice property" (necessary for greedy specifically, and much rarer).
- Sorting activities by start time instead of end time for activity selection — sorting by start time does not produce the correct greedy strategy.
- Assuming a greedy algorithm that works on a small example generalizes correctly, without testing a potential adversarial counterexample.
- Reaching for greedy as a default "simpler" first attempt on a problem that's actually 0/1 knapsack or a DP variant, wasting time before realizing greedy fails on it.

Key Takeaways

- Greedy requires both optimal substructure AND the greedy choice property — the second is the stricter, rarer requirement that makes greedy inapplicable to many problems DP can still solve.
- Activity selection's greedy strategy is sort by earliest finish time, then pick every activity that doesn't conflict with the last one picked — provably optimal via an exchange argument.
- Coin change is the canonical counterexample showing greedy can fail — {1,3,4} making change for 6 is the standard illustration.
- Huffman coding is a real-world, still-used greedy algorithm underlying common compression formats.
- An exchange argument justifies a greedy choice by showing any optimal solution can be transformed to include that choice without getting worse.
- When in doubt, try to construct a counterexample to a proposed greedy strategy before committing to it — finding one means you need DP instead.

Learn More

GeeksforGeeks — Greedy Algorithms

<https://www.geeksforgeeks.org/dsa/greedy-algorithms/>

GeeksforGeeks — Activity Selection Problem

<https://www.geeksforgeeks.org/dsa/activity-selection-problem-greedy-algo-1/>

GeeksforGeeks — Huffman Coding

<https://www.geeksforgeeks.org/dsa/huffman-coding-greedy-algo-3/>